



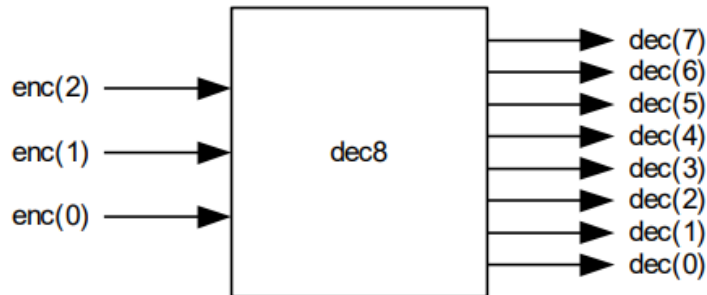
Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ψηφιακά Συστήματα VLSI
Εργαστήριο 1

ΟΝΟΜΑΤΕΠΩΝΥΜΑ	Ευαγγελία Αγγελάκη (el21037) Μαρία Φατούρου(el21077)
ΕΞΑΜΗΝΟ	8ο
ΟΜΑΔΑ	<u>Βάρδια Β/Ομάδα 17</u>

Θέμα A.2: Δυαδικός αποκωδικοποιητής 3 σε 8

Στην άσκηση A2 μας ζητείται να υλοποιήσουμε έναν δυαδικό αποκωδικοποιητή 3 σε 8. Πρόκειται για ένα συνδυαστικό κύκλωμα που ενεργοποιεί διαφορετική έξοδο (μία από τις 8) για κάθε διαφορετικό συνδυασμό των 3 εισόδων.



Αρχικά υλοποιούμε τον αποκωδικοποιητή σε behavioral αρχιτεκτονική. Η behavioral αρχιτεκτονική αφορά την μοντελοποίηση της συμπεριφοράς του κυκλώματος μέσω αλγορίθμων και διαδικασιών, αντί για την ακριβή περιγραφή της δομής του ή της ροής και της σχέσης των δεδομένων. Παρακάτω παρατίθεται ο κώδικας που περιγράφει τον decoder στη behavioral αρχιτεκτονική.

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity decoder3_8_bhv is
```

```
    Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);
```

```
          dec : out STD_LOGIC_VECTOR (7 downto 0));
```

```
end decoder3_8_bhv;
```

```
architecture dec_bhv of decoder3_8_bhv is
```

```
begin
```

```
    process
```

```
    begin
```

```
        case enc is
```

```
            when "000" => dec <= "00000001";
```

```
            when "001" => dec <= "00000010";
```

```
            when "010" => dec <= "00000100";
```

```
            when "011" => dec <= "00001000";
```

```
            when "100" => dec <= "00010000";
```

```
            when "101" => dec <= "00100000";
```

```
            when "110" => dec <= "01000000";
```

```
            when "111" => dec <= "10000000";
```

```
            when others => dec <= "00000000";
```

```
        end case;
    end process;
end dec_bhv;
```

Δημιουργούμε κατάλληλο test bench ώστε να τρέξουμε την προσομοίωση του παραπάνω κώδικα ώστε να επαληθεύσουμε τη λειτουργία του decoder:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity decoder3_8_bhv_tb is
```

```
end decoder3_8_bhv_tb;
```

```
architecture Behavioral of decoder3_8_bhv_tb is
```

```
    component decoder3_8_bhv Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);
        dec : out STD_LOGIC_VECTOR (7 downto 0));
    end component;
    signal enc_tb: STD_LOGIC_VECTOR (2 downto 0);
    signal dec_tb: STD_LOGIC_VECTOR (7 downto 0);
```

```
begin
```

```
DUT: decoder3_8_bhv port map (enc_tb, dec_tb);
```

```
process
```

```
begin
```

```
    enc_tb <= "000";
```

```
    wait for 100ns;
```

```
    enc_tb <= "001";
```

```
    wait for 100ns;
```

```
    enc_tb <= "010";
```

```
    wait for 100ns;
```

```
    enc_tb <= "011";
```

```
    wait for 100ns;
```

```
    enc_tb <= "100";
```

```
    wait for 100ns;
```

```
    enc_tb <= "101";
```

```
    wait for 100ns;
```

```
    enc_tb <= "110";
```

```
    wait for 100ns;
```

```
    enc_tb <= "111";
```

```
    wait for 100ns;
```

```
end process;  
end Behavioral;
```

Στη συνέχεια, η αρχιτεκτονική dataflow είναι ένας τρόπος περιγραφής του κυκλώματος που εστιάζει στη ροή των δεδομένων μεταξύ των σημάτων μέσω λογικών πράξεων και αναθέσεων και όχι μέσω διαδικασιών. Παραθέτουμε την υλοποίηση του decoder σε αρχιτεκτονική dataflow.

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity decoderdataflow is  
    Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);  
          dec : out STD_LOGIC_VECTOR (7 downto 0));  
end decoderdataflow;
```

```
architecture dataflow of decoderdataflow is  
begin  
    dec(0) <= (not enc(0)) and (not enc(1)) and (not enc(2));  
    dec(1) <= (enc(0) and (not enc(1)) and (not enc(2)));  
    dec(2) <= ((not enc(0)) and enc(1) and (not enc(2)));  
    dec(3) <= (enc(0) and enc(1) and (not enc(2)));  
    dec(4) <= ((not enc(0)) and (not enc(1)) and enc(2));  
    dec(5) <= (enc(0) and (not enc(1)) and enc(2));  
    dec(6) <= ((not enc(0)) and enc(1) and enc(2));  
    dec(7) <= enc(0) and enc(1) and enc(2);  
end dataflow;
```

και κατάλληλο test bench που να διατρέχει όλες τις πιθανές τιμές της εισόδου:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```

entity decoder_tb is
-- Port ( );
end decoder_tb;

architecture dataflow of decoder_tb is
component decoderdataflow Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);
    dec : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    signal enc_tb: STD_LOGIC_VECTOR (2 downto 0);
    signal dec_tb: STD_LOGIC_VECTOR (7 downto 0);
    signal clk_tb:std_logic;
begin
    DUT: decoderdataflow port map (enc_tb, dec_tb);
process
begin
enc_tb <= "000";
wait for 100ns;
enc_tb <= "001";
wait for 100ns;
enc_tb <= "010";
wait for 100ns;
enc_tb <= "011";
wait for 100ns;
enc_tb <= "100";
wait for 100ns;
enc_tb <= "101";
wait for 100ns;
enc_tb <= "110";
wait for 100ns;
enc_tb <= "111";
wait for 100ns;

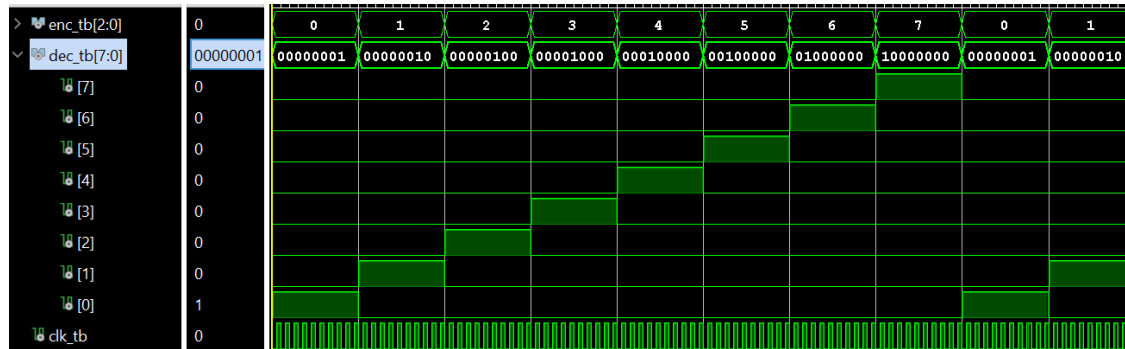
end process;

generate_clock:process
begin
    clk_tb<='0';
    wait for 5ns;
    clk_tb<='1';
    wait for 5ns;
end process;

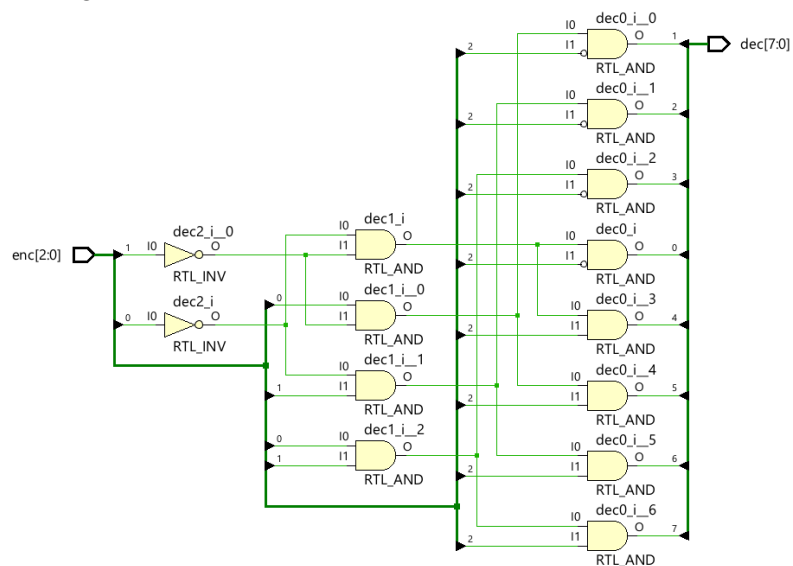
```

end dataflow;

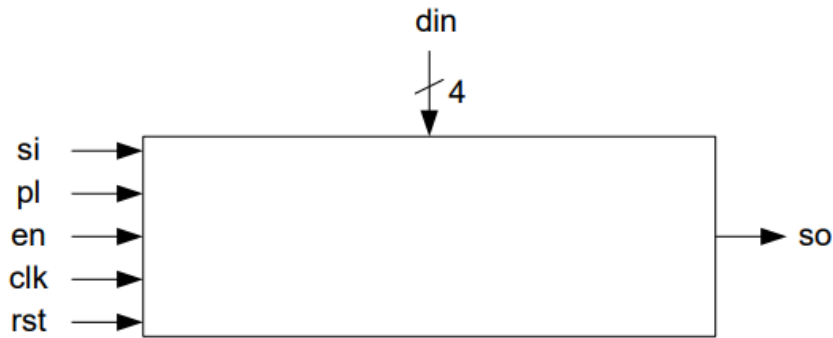
Η λειτουργία του decoder επαληθεύεται μέσω της προσομοίωσης



και το elaborated design:



Θέμα Β.2: Καταχωρητής ολίσθησης των 4 bits με παράλληλη φόρτωση



Σχ. 2.7: Καταχωρητής δεξιάς ολίσθησης των 4 bits

Εμπλουτίζουμε το πρόγραμμα που μας δίνεται και σχεδιάζουμε έναν καταχωρητή δεξιάς και αριστερής ολίσθησης. Η φορά της ολίσθησης εξαρτάται από τη μεταβλητή choose. Όταν το choose είναι 0 έχουμε δεξιά ολίσθηση ενώ όταν είναι 1 έχουμε αριστερή. Παράλληλα, όταν reset=0 η είσοδος μηδενίζεται, στην ακμή του ρολογιού με pl=1 έχουμε φόρτωση της νέας εισόδου, ενώ όταν έχουμε ακμή του ρολογιού και enable=1 έχουμε ολίσθηση. Στην περίπτωση της αριστερής ολίσθησης το κρατούμενο εξόδου(so) είναι το MSB και στην περίπτωση της δεξιάς το LSB.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity shift_register is
  Port ( clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        si : in STD_LOGIC;
        en : in std_logic;
        pl : in STD_LOGIC;
        choose : in STD_LOGIC;
        din : in STD_LOGIC_VECTOR (3 downto 0);
        so : out STD_LOGIC);
end shift_register;
```

```
architecture Behavioral of shift_register is
  signal dff: std_logic_vector(3 downto 0);
begin
  edge: process (clk, rst)
  begin
    if rst = '0' then
      dff <= (others => '0');
    elsif clk'event and clk = '1' then
      if pl = '1' then
        dff <= din;
      end if
    end if
  end process
end Behavioral;
```

```

    elsif en = '1' then
        case choose is
            when '0' =>
                dff <= si & dff(3 downto 1);
            when others =>
                dff <= dff(2 downto 0) & si;
            end case;
        end if;
    end if;
end process;
with choose select
    so <= dff(0) when '0',
        dff(3) when others;

```

end Behavioral;

Ο κώδικας για το test bench:

```

entity shift_reg_tb is
    -- Port ( );
end shift_reg_tb;

```

```

architecture Behavioral of shift_reg_tb is
    component shift_register Port ( clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        si : in STD_LOGIC;
        en : in std_logic;
        pl : in STD_LOGIC;
        choose : in STD_LOGIC;
        din : in STD_LOGIC_VECTOR (3 downto 0);
        so : out STD_LOGIC);
    end component;
    signal clk_tb: std_logic;
    signal rst_tb: std_logic;
    signal si_tb: std_logic;
    signal en_tb: std_logic;
    signal pl_tb: std_logic;

```

```

signal choose_tb: std_logic;
signal din_tb: std_logic_vector(3 downto 0);
signal so_tb: std_logic;

begin
DUT: shift_register port map (clk_tb,rst_tb,si_tb,en_tb,pl_tb,choose_tb,din_tb,so_tb);
stimulus:process
begin
rst_tb <= '1';
en_tb <= '0';
si_tb <= '0';
pl_tb <= '1';
choose_tb <= '0';
din_tb <= "0101";
wait for 10ns;

pl_tb <= '0';
choose_tb <= '0';
si_tb <= '1';
en_tb <= '1';
wait for 10ns;

si_tb <= '0';
wait for 40ns;

pl_tb <= '0';
choose_tb <= '1';
si_tb <= '1';
en_tb <= '1';
wait for 40ns;

rst_tb <= '0';
choose_tb <= '0';
si_tb <= '0';
wait for 10ns;

rst_tb <= '1';
en_tb <= '0';
si_tb <= '0';
pl_tb <= '1';
choose_tb <= '0';
din_tb <= "0001";

```

```

wait for 10ns;
pl_tb <= '0';
wait for 20ns;
en_tb <= '1';
wait for 20ns;

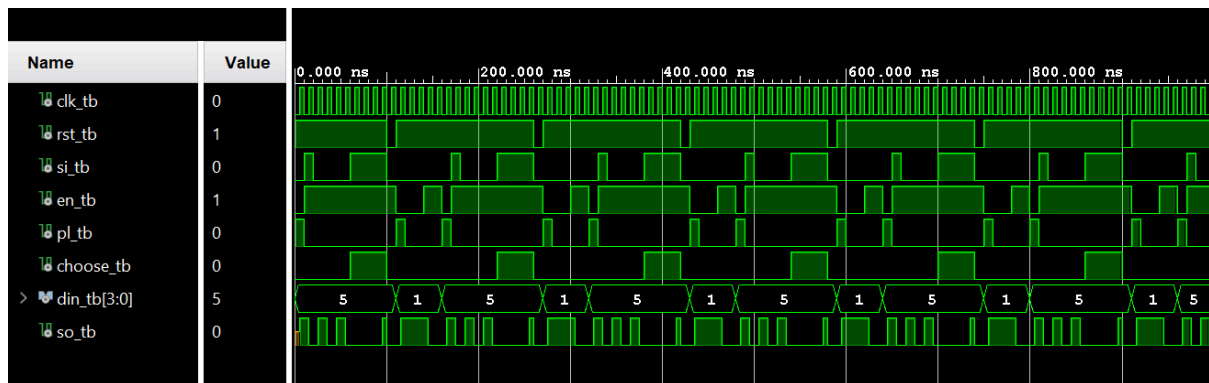
end process;

generate_clock:process
begin
  clk_tb<='0';
  wait for 5ns;
  clk_tb<='1';
  wait for 5ns;
end process;

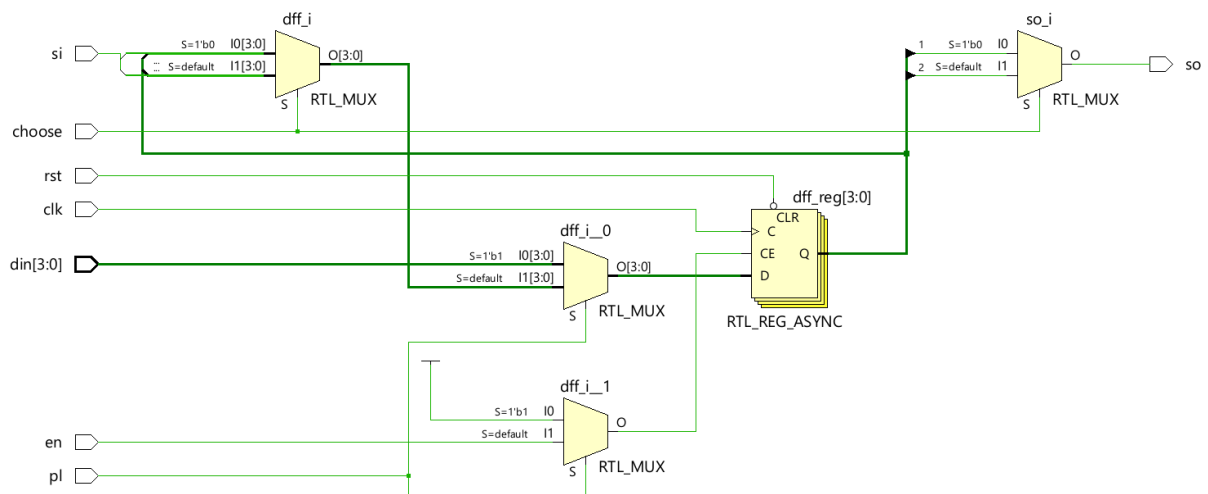
end Behavioral;

```

Παραθέτουμε την προσομοίωση:



και το elaborated design:



Θέμα Β.3: Μετρητής 3 bit με είσοδο ενεργοποίησης και κρατούμενο εξόδου

Στην παρούσα άσκηση σχεδιάζουμε σε VHDL έναν μετρητή που μετράει προς τα πάνω μέχρι το 7 και προς τα κάτω μέχρι το 0. Κατά προτεραιότητα όταν το resetn γίνεται 0, μηδενίζεται το count, ενώ μέτρηση πραγματοποιείται μόνο όταν το count_en(enable) είναι ίσο με 1. Η επιλογή φοράς της μέτρησης υλοποιείται μέσω της μεταβλητής mode. Όταν mode=1 πραγματοποιείται μέτρηση προς τα πάνω ενώ όταν mode=0 μετράμε προς τα κάτω. Η επιλογή αυτή υλοποιείται με case-when, γεγονός που καθιστά τη συγκεκριμένη αρχιτεκτονική behavioral. Η έξοδος της μέτρησης φαίνεται στη μεταβλητή sum που είναι τύπου std_logic_vector.

Ο κώδικας της άσκησης:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity counter3bit is
  Port (clk:in std_logic;
        mode:in std_logic;
        resetn:in std_logic;
        count_en : in std_logic;
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic);
end counter3bit;

architecture Behavioral of counter3bit is
  signal count : std_logic_vector(2 downto 0);
begin
  process(clk, resetn)
  begin
    if resetn='0' then --ασυγχρονος μηδενισμος
      count <= (others=>'0');
    elsif clk'event and clk='1' then
      if count_en = '1' then -- Μέτρηση μόνο αν count_en='1'
        case mode is
```

```

    when '0' => count<=count-1;
    when '1' =>
        if (count /= 7) then
            count <= count + 1;
        end if;
        when others =>count<=(others=>'0');
    end case;
end if;
end if;
end process;
sum<= count;
cout <= '1' when count=7 and count_en='1' else '0';

```

end Behavioral;

Δημιουργούμε κατάλληλο test bench πρόγραμμα για να επαληθεύσουμε τη λειτουργία του κυκλώματος:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity counter3bit_tb is
    -- Port ( );
end counter3bit_tb;

architecture Behavioral of counter3bit_tb is
    component counter3bit is
        Port (clk:in std_logic;
            mode:in std_logic;
            resetn:in std_logic;
            count_en : in std_logic;
            sum : out std_logic_vector(2 downto 0);
            cout : out std_logic);
    end component;
    signal clk_tb: std_logic:='0';
    signal mode_tb: std_logic:='1';
    signal resetn_tb: std_logic:='0';
    signal count_en_tb: std_logic:='0';

```

```

signal sum_tb:std_logic_vector(2 downto 0);
signal cout_tb: std_logic;

begin
DUT: counter3bit port map (clk_tb,mode_tb,resetn_tb,count_en_tb,sum_tb,cout_tb);

stimulus:process
begin
    resetn_tb<='1';
    count_en_tb<='1';
    clk_tb<='1';
    mode_tb<='1';
    for i in 0 to 4 loop
        clk_tb<=not clk_tb;
        wait for 10ns;
    end loop;

    count_en_tb<='0';
    for i in 0 to 9 loop
        clk_tb<=not clk_tb;
        wait for 10ns;
    end loop;

    mode_tb<='1';
    count_en_tb<='1';
    for i in 0 to 17 loop
        clk_tb <= not clk_tb;
        wait for 10ns;
    end loop;

    count_en_tb<='0';
    for i in 0 to 9 loop
        clk_tb <= not clk_tb;
        wait for 10ns;
    end loop;

    count_en_tb<='1';
    mode_tb<='0';
    for i in 0 to 17 loop
        clk_tb<=not clk_tb;
        wait for 10ns;
    end loop;
end process;

```

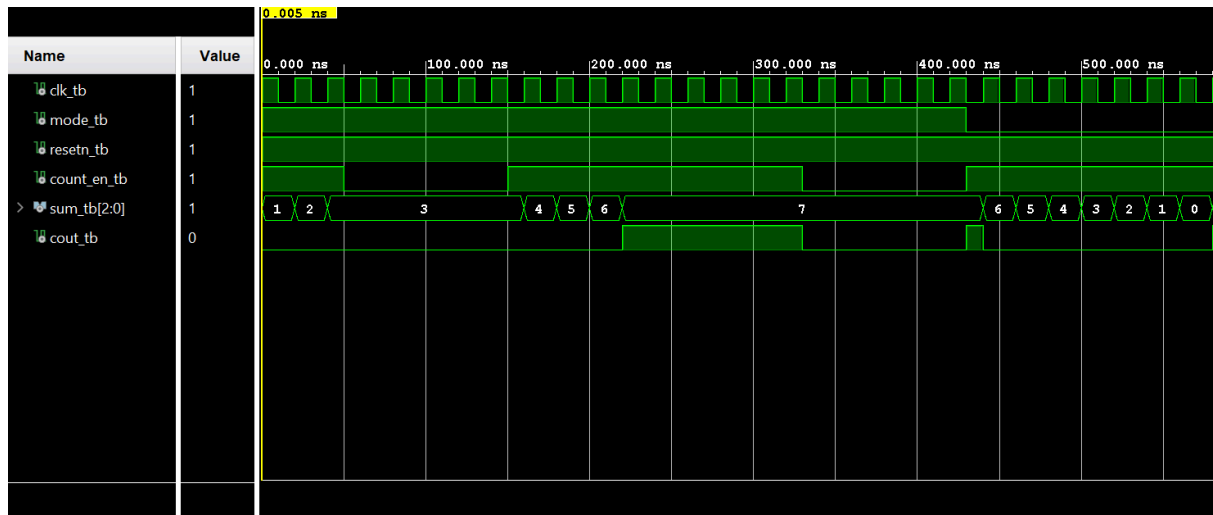
```

resetsn_tb<='0';
for i in 0 to 17 loop
clk_tb<=not clk_tb;
    wait for 10ns;
end loop;
wait;
end process;

```

end Behavioral;

Παραθέτουμε το simulation στο οποίο επιβεβαιώνεται η ζητούμενη λειτουργία του μετρητή:



Με το “open elaborated design” του vivado λαμβάνουμε το εξής σχήμα:

